



James

Posted on Apr 22, 2024



37

Mastering LangChain: Part 1 - Introduction to LangChain and Its Key Components

#ai #python #langchain #llm

LangChain (4 Part Series)

- 1 Mastering LangChain: Part 1 - Introduction to LangCh...**
- 2 Part 2: Mastering Prompts and Language Models with Lan...
- 3 Part 3: Building Powerful Chains and Agents in LangChain
- 4 LangChain Part 4 - Leveraging Memory and Storage in La...

Part 1: Introduction to LangChain and Its Key Components

Welcome to the first part of our in-depth tutorial series on mastering LangChain, a powerful Python library for building applications with large language models (LLMs). In this tutorial, we'll introduce you to LangChain and its essential components and guide you through setting up your development environment to start working with this incredible tool.

1. What is LangChain?

1.1. Overview of the library and its purpose

LangChain is an open-source Python library that simplifies the process of building applications with LLMs. It provides tools and abstractions to help you integrate LLMs into your projects, create robust chains and agents, and manage memory and storage.

The primary goal of LangChain is to offer a high-level, modular, and extensible framework that allows developers to focus on building their application logic while handling the complexities of interacting with LLMs behind the scenes. LangChain supports various LLM providers, such as OpenAI, Hugging Face, and more, making it a versatile tool for working with different models.

1.2. Benefits of using LangChain in your projects

Incorporating LangChain into your projects offers several key benefits:

1. **Simplified LLM integration:** LangChain provides a consistent and intuitive interface for working with various LLMs. It abstracts away the details of interacting with specific models or providers, allowing you to easily switch between different LLMs without modifying your application code extensively.
2. **Modular and reusable components:** The library's modular design enables you to create and combine different components, such as prompts, chains, and agents, to build complex applications. These components are highly reusable, promoting code efficiency and maintainability.
3. **Extensibility:** LangChain is designed with extensibility in mind. You can seamlessly create custom components or integrate them with other libraries and tools. This flexibility allows you to tailor LangChain to your specific use case and leverage the existing ecosystem.
4. **Growing community and rich ecosystem:** LangChain has a thriving community of developers contributing to its development, sharing knowledge, and creating extensions. The library also offers a rich ecosystem of examples, tutorials, and resources to help you get started and tackle advanced use cases effectively.

2. Setting up LangChain

2.1. Installing LangChain and its dependencies

To start using LangChain, you must install the library and its dependencies. You can install LangChain using pip, the Python package installer:

```
pip install langchain
```

In addition to the core library, you may need to install specific dependencies for the LLMs or other components you plan to use. For example, if you want to use OpenAI's GPT models, you'll need to install the `openai` package:

```
pip install openai
```

For any additional installation requirements, refer to the documentation of the specific LLMs or components you intend to use.

2.2. Configuring your development environment

To use LangChain effectively, you must set up your development environment with the necessary API keys and configurations. Many LLM providers require authentication to access their APIs. For instance, if you're using OpenAI's models, you'll need to obtain an API key from the OpenAI website and set it as an environment variable:

```
export OPENAI_API_KEY="your-api-key"
```

Replace `"your-api-key"` with your actual OpenAI API key. It is crucial to keep your API keys secure and avoid sharing them publicly. Consider using environment variables or secure configuration files

to store sensitive information.

3. Understanding the Core Components of LangChain

LangChain consists of several core components that work together to build robust applications. Let's explore each of these components in more detail:

3.1. Prompts

Prompts are the input text you provide to an LLM to generate a response. They are crucial in guiding the LLM's output and defining the task. LangChain offers a variety of prompt templates and utilities to help you create effective prompts.

Here's an example of creating a prompt template using LangChain:

```
from langchain import PromptTemplate

template = "What is the capital of {country}?"
prompt = PromptTemplate(template=template, input_variables=["country"])
```

In this example, we define a prompt template that asks for the capital of a given country. The `{country}` placeholder indicates where the country name will be inserted. We then create a `PromptTemplate` instance, specifying the template and the input variables it expects.

3.2. Language Models (LLMs)

Language Models (LLMs) are the core engines behind LangChain applications. They are responsible for generating human-like text based on the input prompts. LangChain supports various LLMs, including OpenAI's GPT models, Hugging Face models, and more.

Here's an example of initializing an OpenAI LLM using LangChain:

```
from langchain.llms import OpenAI

llm = OpenAI(model_name="text-davinci-002", temperature=0.7)
```

In this example, we create an instance of the OpenAI LLM, specifying the model name ("text-davinci-002") and the temperature parameter, which controls the randomness of the generated output.

3.3. Chains

Chains allow you to combine multiple components, such as prompts and LLMs, to create more complex applications. They define a sequence of steps to process input, generate output, and perform additional tasks. LangChain provides a variety of built-in chains and supports the creation of custom chains.

Here's an example of creating an LLMChain using LangChain:

```
from langchain.chains import LLMChain

chain = LLMChain(llm=llm, prompt=prompt)
result = chain.run("United States")
print(result)
```

In this example, we create an instance of `LLMChain`, passing the previously defined `llm` and `prompt` instances. We then use the `run()` method to execute the chain with the input "United States." The resulting output, which is the capital of the United States, is printed.

3.4. Agents

Agents are high-level abstractions that use chains and tools to accomplish specific goals. They can make decisions, interact with external tools, and retrieve information to complete tasks. Agents are particularly useful for building conversational AI applications or automating complex workflows.

Here's an example of creating an agent using LangChain:

```
from langchain.agents import load_tools, initialize_agent

tools = load_tools(["serpapi", "llm-math"], llm=llm)
agent = initialize_agent(tools, llm, agent="zero-shot-react-description", verbose=True)
result = agent.run("What is the population of Paris, France?")
print(result)
```

In this example, we load a set of tools ("serpapi" for web search and "llm-math" for mathematical operations) using the `load_tools()` function. We then initialize an agent with the loaded tools, the LLM instance, and the agent type ("zero-shot-react-description"). The `verbose=True` parameter enables verbose output for debugging purposes. Finally, we use the `run()` method to execute the agent with the input question, "What is the population of Paris, France?". The agent uses the available tools and the LLM to generate a printed response.

3.5. Memory Components in LangChain

Memory components are critical in LangChain for storing and retrieving data across multiple interactions or conversations. These components help applications maintain contextual continuity, which is crucial for building coherent and dynamic dialogues. LangChain offers a variety of memory implementations to cater to different operational needs, such as `ConversationBufferMemory` and `ConversationSummaryMemory`.

Using ConversationBufferMemory:

This memory model is designed to capture and recall detailed interaction logs. It is ideal for applications that require a precise history of user exchanges to provide contextually relevant responses. Below is an example of its usage:

```
from langchain.memory import ConversationBufferMemory

# Initialize memory buffer
memory = ConversationBufferMemory()

# Storing conversation context
memory.save_context({"input": "Hi"}, {"output": "Hello! How can I assist you today?"})
memory.save_context({"input": "What's the weather like?"}, {"output": "I apologize, but I don't
```

In this example, `save_context()` stores pairs of user inputs and system outputs. This type of memory is essential for systems that reference previous interactions during a session.

Using ConversationSummaryMemory:

Alternatively, `ConversationSummaryMemory` provides a way to retain a condensed version of conversations. This is useful for applications that need to understand the essence of previous interactions without the overhead of detailed transaction logs. Here's how you might implement it:

```
from langchain.memory import ConversationSummaryMemory

# Initialize summary memory
memory = ConversationSummaryMemory()

# Summarizing key conversation points
```

```
memory.save_summary("user_greeting", "User greeted the system.")  
memory.save_summary("weather_inquiry", "User asked about the weather but was informed of the la
```

This approach allows the application to quickly recap the critical elements of a conversation, facilitating smoother transitions and more informed responses in ongoing interactions.

Conclusion

This introductory post outlines the foundation of LangChain. Upcoming entries will delve into each component with advanced examples and practical applications. Join us next time as we explore the art of crafting effective prompts and integrating diverse language models.

LangChain (4 Part Series)

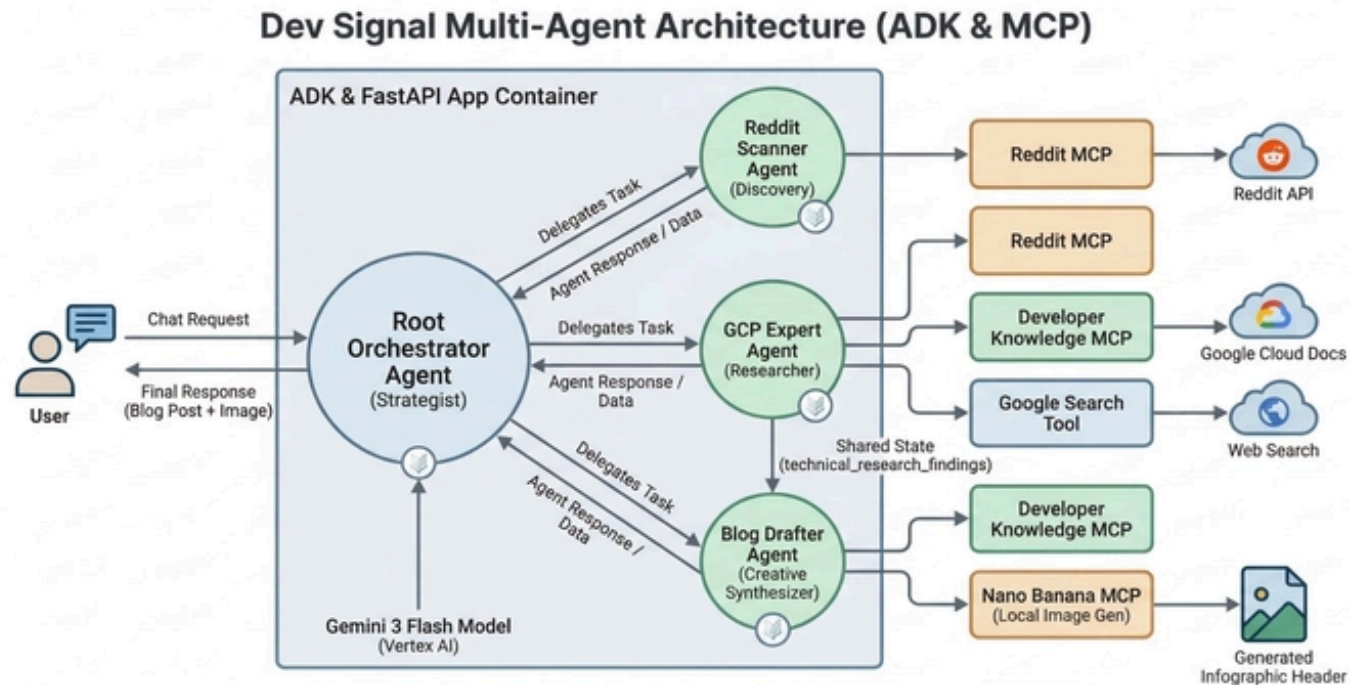
- 1 **Mastering LangChain: Part 1 - Introduction to LangCh...**
- 2 Part 2: Mastering Prompts and Language Models with Lan...
- 3 Part 3: Building Powerful Chains and Agents in LangChain
- 4 LangChain Part 4 - Leveraging Memory and Storage in La...



The DEV Team

PROMOTED





Architect A Personalized Multi-Agent System with Long-Term Memory

In support of our mission to accelerate the developer journey on Google Cloud, we built Dev Signal a multi-agent system designed to transform raw community signals into reliable technical

[Read More](#)

Top comments (1)



Kevin • Dec 19 '24

...



Really enjoyed reading your comprehensive guide on LangChain! As someone who's been exploring AI tools, I wanted to share some additional benefits/thoughts that might help others who are just starting out.

Making AI More Human-Friendly

While the tutorial covers the technical aspects really well, I've found that LangChain is particularly good at making AI feel more natural and human-like. For example, you can set it up to:

- Remember previous conversations (like how a human would)
- Understand the tone of the conversation and adjust accordingly
- Take breaks between responses to seem more natural
- Remember user preferences over time

While other AI tools on the market can do this, I find Langchain is particularly adept at this.

Real-World Uses I've Discovered

I've seen some really creative uses of LangChain that weren't mentioned in the article:

- Virtual tutors that adapt to students' learning styles
- Customer service bots that can actually understand emotion in questions
- Content creators using it to brainstorm ideas while maintaining their unique voice
- Research assistants that can pull information from multiple sources and make it make sense

One Thing I Wish I Knew At The Start

One thing I wish someone had told me early on: LangChain can feel overwhelming at first! It's like learning a new language - you don't need to understand everything right away (and you won't).

Start small, maybe with simple chat responses, and build up from there. The community is super helpful, and there are lots of resources for beginners.

A Cool Trick I Learned

Here's something neat I discovered: you can use LangChain to create different "personalities" for different purposes. For example, you might want a professional tone for business emails but a more casual voice for social media. LangChain makes this really easy to switch between.

Looking Ahead

The most exciting part about LangChain is how it keeps growing and improving. Every month there seem to be new features that make it easier to use and more powerful. It's great for people who want to work with AI but don't want to get bogged down in complex technical details.

I think the most valuable lesson I've learned is that you don't need to be a technical expert to benefit from LangChain. It's more about understanding what you want to achieve and then using the right tools to get there.

For a more technical deep-dive into LangChain, I highly recommend checking out Eduardo Maciel's excellent tutorial on ScalablePath: [A Hands-on Tutorial to Building a Project with Langchain](#). His comprehensive guide covers all the technical aspects I didn't touch on here and is perfect for those ready to dive deeper into the development side of things.

DEV

The DEV Team

PROMOTED

...



Dialling Our Agents to 11: My Favourite MCP Servers

I'd like to tell you about a few of the MCP servers and agent skills that I use most often. I'll tell you what they do, how I use them, and how to install them.

This first part will cover the MCP servers, and I'll share my favourite agent skills in the second part.

[Read more →](#)**James**

If you would like to support my writing or buy me a beer: <https://buymeacoffee.com/bmours>

LOCATION

Denver

EDUCATION

Ohio State - ISE , Georgia institute of technology - Masters in CS

WORK

Freelance Consulting

JOINED

Jan 23, 2024

More from James

Building a Human-in-the-Loop AI App with LangGraph and Ollama

#python #ai #llm #langgraph

Building a News and Stock Assistant with LangGraph and Chainlit

#programming #ai #tutorial #python

Mastering MCP Servers with LangChain and LangGraph: A Beginner's Guide

#python #programming #langgraph #ai



James

Posted on Jun 26, 2024



7

Part 2: Mastering Prompts and Language Models with LangChain

#python #llm #langchain #vectordatabase

LangChain (4 Part Series)

- 1 Mastering LangChain: Part 1 - Introduction to LangChain ...
- 2 Part 2: Mastering Prompts and Language Models with...**
- 3 Part 3: Building Powerful Chains and Agents in LangChain
- 4 LangChain Part 4 - Leveraging Memory and Storage in La...

Part 2: Mastering Prompts and Language Models with LangChain

In the previous part of our LangChain tutorial series, we introduced the core components of the library. Now, let's dive deeper into two essential aspects of building LangChain applications: prompts and language models (LLMs). You'll learn how to create effective prompts, integrate various LLMs, and customize them for your specific use cases.

Introduction to Prompt Engineering

Prompt engineering is the art of designing and crafting input prompts that guide the behavior of LLMs to generate desired outputs. Well-crafted prompts are crucial for obtaining high-quality and relevant responses from LLMs.

LangChain provides the `PromptTemplate` class to create structured and dynamic prompts. With `PromptTemplate`, you can define a template string that includes input variables, which can be dynamically populated with specific values when the prompt is executed.

Example: Creating a Basic Prompt Template

```
from langchain import PromptTemplate

template = "Translate the following English text to {target_language}: {text}"
prompt = PromptTemplate(template=template, input_variables=["target_language", "text"])

# Using the prompt with input values
formatted_prompt = prompt.format(target_language="Spanish", text="Hello, how are you?")
print(formatted_prompt)
```

This would output:

```
Translate the following English text to Spanish: Hello, how are you?
```

Designing Effective Prompts

To create effective prompts, consider the following best practices and principles:

1. **Clarity and Specificity:** Be clear and specific about the task or question you want the LLM to address.
2. **Dynamic and Reusable:** Use input variables to make your prompts dynamic and reusable across different inputs.

3. **Context and Examples:** Provide sufficient context and examples to guide the LLM towards the desired output format and style.
4. **Experimentation:** Experiment with different prompt variations and assess their impact on the generated responses.

Example: Summarization Prompt

```
from langchain import PromptTemplate

summarization_template = "Summarize the following text in {num_sentences} sentences: {text}"
summarization_prompt = PromptTemplate(template=summarization_template, input_variables=["num_sen

# Using the summarization prompt
formatted_prompt = summarization_prompt.format(num_sentences=3, text="Artificial intelligence i
print(formatted_prompt)
```

Example: Question-Answering Prompt

```
from langchain import PromptTemplate

qa_template = "Answer the following question based on the provided context:\nContext: {context}
qa_prompt = PromptTemplate(template=qa_template, input_variables=["context", "question"])

# Using the question-answering prompt
formatted_prompt = qa_prompt.format(context="The Eiffel Tower is located in Paris, France.", q
print(formatted_prompt)
```

Example: Translation Prompt

```
from langchain import PromptTemplate

translation_template = "Translate the following text from {source_language} to {target_language}"
translation_prompt = PromptTemplate(template=translation_template, input_variables=["source_language", "target_language"])

# Using the translation prompt
formatted_prompt = translation_prompt.format(source_language="English", target_language="French")
print(formatted_prompt)
```

Integrating Language Models

LangChain supports various LLMs, including OpenAI, Hugging Face, Cohere, and more. To integrate an LLM into your LangChain application, you need to create an instance of the corresponding LLM class.

Example: Integrating OpenAI

```
from langchain.llms import OpenAI

# Creating an instance of the OpenAI LLM
llm = OpenAI(model_name="text-davinci-002", temperature=0.7)

# Using the LLM with a prompt
response = llm("Translate the following English text to Spanish: Hello, how are you?")
print(response)
```

When selecting an LLM, consider factors such as the specific task requirements, model capabilities, performance, and cost. LangChain provides a consistent interface across different LLMs, making it

easy to switch between them as needed.

Example: Integrating Hugging Face

```
from langchain.llms import HuggingFacePipeline
from transformers import AutoModelForCausalLM, AutoTokenizer

# Creating an instance of the Hugging Face LLM
model_id = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id)
llm = HuggingFacePipeline(model=model, tokenizer=tokenizer)

# Using the LLM with a prompt
response = llm("Translate the following English text to French: Good morning!")
print(response)
```

Customizing and Fine-Tuning LLMs

In some cases, you may want to fine-tune an LLM to improve its performance on a specific task or domain. Fine-tuning involves training the LLM on a smaller dataset relevant to your use case, allowing it to adapt to the specific language patterns and knowledge required.

Example: Fine-Tuning with Hugging Face

```
from langchain.llms import HuggingFacePipeline
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments, Trainer
from datasets import load_dataset

# Load a dataset
dataset = load_dataset('text', data_files={'train': 'path/to/your/train.txt', 'test': 'path/to/
```

```
# Fine-tuning configurations
training_args = TrainingArguments(
    output_dir="./results",
    evaluation_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=4,
    num_train_epochs=3,
    weight_decay=0.01,
)

# Fine-tune the model
model_id = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id)
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset['train'],
    eval_dataset=dataset['test'],
)
trainer.train()

# Integrate the fine-tuned model
llm = HuggingFacePipeline(model=model, tokenizer=tokenizer)

# Using the fine-tuned model
response = llm("Translate the following English text to German: Good morning!")
print(response)
```

Advanced Prompting Techniques

LangChain offers advanced prompting techniques to enhance the capabilities of your LLMs. One such technique is incorporating external knowledge using document loaders and vector stores.

Document Loaders and Vector Stores

Document loaders allow you to load and process external data sources, such as web pages, PDF files, or databases. Vector stores enable efficient similarity search over the loaded documents, helping you retrieve relevant information based on the input query.

```
from langchain.document_loaders import WebBaseLoader
from langchain.indexes import VectorstoreIndexCreator

# Loading documents from a webpage
loader = WebBaseLoader("https://example.com")
index = VectorstoreIndexCreator().from_loaders([loader])

# Querying the index
query = "What are the key features of the product?"
docs = index.similarity_search(query)
for doc in docs:
    print(doc)
```

Controlling Output Style and Structure

Another advanced technique is controlling the output style, structure, and length of the generated responses. You can achieve this by providing examples or templates within your prompts that demonstrate the desired output format.

```
from langchain import PromptTemplate
```

```
product_description_template = """
Generate a product description based on the following information:
Product: {product_name}
Key features: {features}
Competitor analysis: {competitor_info}

Product Description:
"""

prompt = PromptTemplate(template=product_description_template, input_variables=["product_name",

# Using the prompt with detailed input
formatted_prompt = prompt.format(
    product_name="Super Blender",
    features="High-speed motor, Multiple blending modes",
    competitor_info="Similar products in the market lack advanced features."
)
print(formatted_prompt)
```

Managing Prompts and Responses

When working with LLMs, it's important to handle prompts and responses effectively. LangChain provides methods to send prompts to LLMs and retrieve the generated responses.

Example: Sending Prompts and Handling Responses

```
# Using the OpenAI LLM with a detailed prompt
response = llm(formatted_prompt)
print(response)
```

To ensure robustness, implement error-handling mechanisms to handle scenarios such as timeouts or invalid responses. LangChain provides utilities for parsing and processing LLM output, making it easier to extract relevant information for downstream tasks.

Example: Parsing Responses with Regex

```
from langchain.output_parsers import RegexParser

# Define a regex parser to extract the product description
parser = RegexParser(regex=r"Product Description:\s*(.*)")
parsed_output = parser.parse(response)
print(parsed_output)
```

In the next part of this tutorial series, we'll explore how to combine prompts and LLMs with chains and agents to build more complex and interactive applications.

Stay tuned for Part 3, where we'll dive into the world of chains and agents in LangChain!

Leave comments with what you would like me to cover next.

If you would like to support me or buy me a beer feel free to join my Patreon [jamesbmour](#)

LangChian (4 Part Series)

1 Mastering LangChain: Part 1 - Introduction to LangChain ...

2 Part 2: Mastering Prompts and Language Models with...

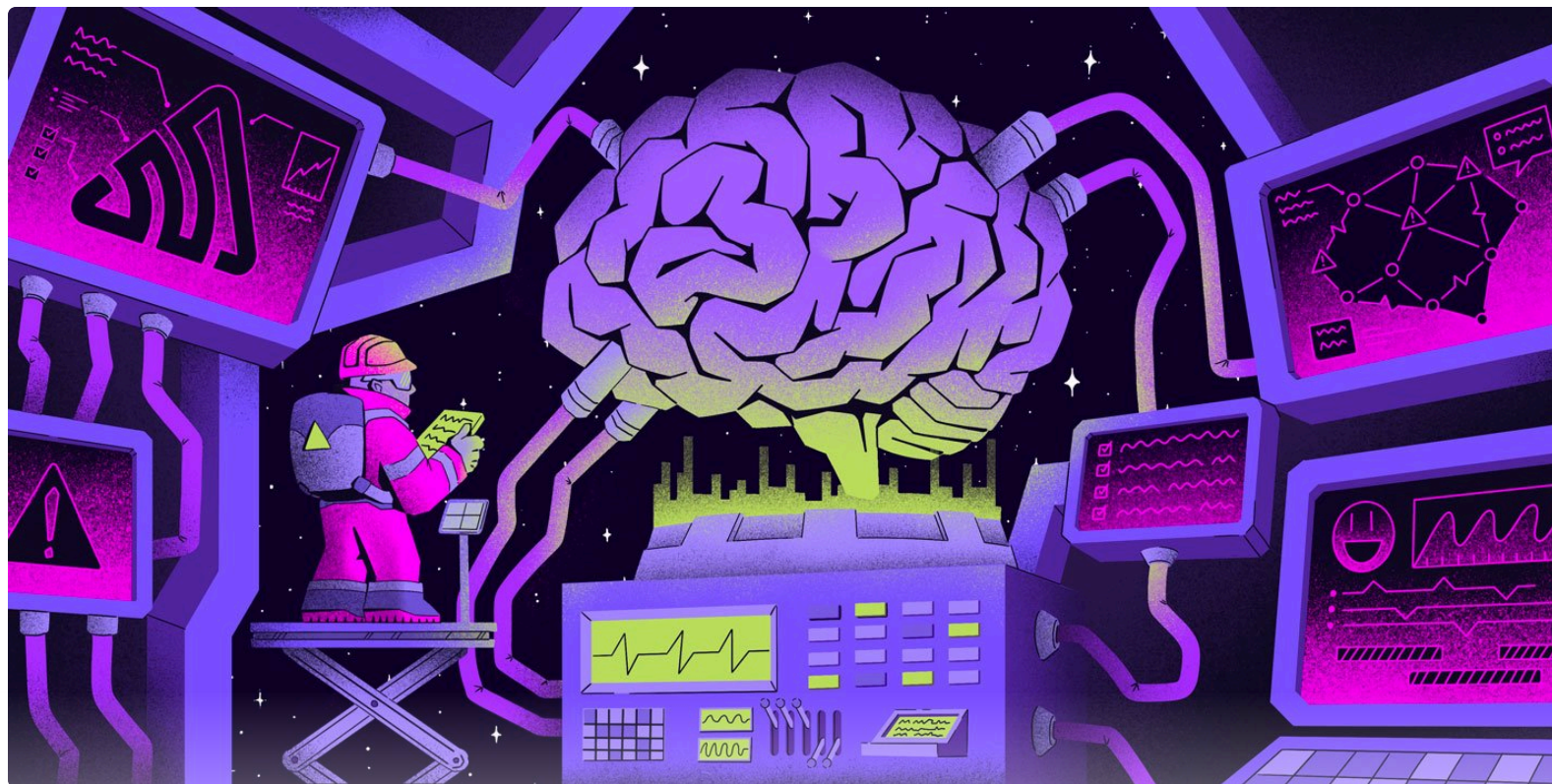
3 Part 3: Building Powerful Chains and Agents in LangChain

4 LangChain Part 4 - Leveraging Memory and Storage in La...



Sentry

PROMOTED



You launched an MCP server, but are you monitoring it?

Built an MCP server? Now see everything it does. Sentry's MCP Server Monitoring tracks every client, tool, and request so you can fix issues fast and build with confidence.

[Read More](#)

Top comments (0)

[Code of Conduct](#) • [Report abuse](#)



Bitrise

PROMOTED



A purple banner for Bitrise. In the top left is the Bitrise logo, which consists of a white icon of a face with a single dot for a nose and a wide smile, followed by the word "bitrise" in white lowercase letters. Below the logo, the text "The CI/CD built for iOS." is written in a large, white, sans-serif font. Underneath that, "Ready for what's next" is written in a slightly smaller white font. Further down, the phrases "Build faster." and "Ship with confidence." are stacked in a smaller white font. At the bottom left of the banner is a white rounded rectangle containing the text "See it in action today" in a bold, black, sans-serif font. On the right side of the banner is a 3D illustration of a purple smartphone standing on a white, stylized, interlocking ring structure. Several small, colorful icons (a green square, a blue square with a white icon, a green square with a white rocket icon, and a yellow square) are scattered around the base of the ring structure.

bitrise

The CI/CD built for iOS. Ready for what's next

Build faster.
Ship with confidence.

See it in action today

Try Bitrise free and feel the DevOps difference today!

Your mobile team is growing but so is the chaos.

More people. More code. Endless moving parts. That's where Bitrise comes in. Our award-winning Mobile DevOps platform aligns your people, tools, and processes so you can build faster, ship sooner and reignite that joy in mobile dev again.

[Start a free trial](#)

**James**

If you would like to support my writing or buy me a beer: <https://buymeacoffee.com/bmours>

LOCATION

Denver

EDUCATION

Ohio State - ISE , Georgia institute of technology - Masters in CS

WORK

Freelance Consulting

JOINED

Jan 23, 2024

More from James

Building a Human-in-the-Loop AI App with LangGraph and Ollama

#python #ai #llm #langgraph

Building a News and Stock Assistant with LangGraph and Chainlit

#programming #ai #tutorial #python

Mastering MCP Servers with LangChain and LangGraph: A Beginner's Guide

#python #programming #langgraph #ai

**The DEV Team****PROMOTED**



I built a native Android app in an afternoon, and I've never written a line of Kotlin

I've always thought building a mobile app required climbing a massive learning curve just to get a basic environment set up. To test that theory, I tried building my very first Android app using Google AI Studio.

Five minutes later, I had a working prototype.

[Read more →](#)



James

Posted on Jul 31, 2024



15



1



1



1



1

Part 3: Building Powerful Chains and Agents in LangChain

#langchain #python #llm #openai

LangChain (4 Part Series)

1 Mastering LangChain: Part 1 - Introduction to LangChain ...

2 Part 2: Mastering Prompts and Language Models with Lan...

3 Part 3: Building Powerful Chains and Agents in LangC...

4 LangChain Part 4 - Leveraging Memory and Storage in La...

Building Powerful Chains and Agents in LangChain

In this comprehensive guide, we'll dive deep into the world of LangChain, focusing on constructing powerful chains and agents. We'll cover everything from understanding the fundamentals of chains to combining them with large language models (LLMs) and introducing sophisticated agents for autonomous decision-making.

1. Understanding Chains

1.1 What are Chains in LangChain?

Chains in LangChain are sequences of operations or tasks that process data in a specific order. They allow for modular and reusable workflows, making it easier to handle complex data processing and language tasks. Chains are the building blocks for creating sophisticated AI-driven systems.

1.2 Types of Chains

LangChain offers several types of chains, each suited for different scenarios:

1. **Sequential Chains:** These chains process data in a linear order, where the output of one step serves as the input for the next. They're ideal for straightforward, step-by-step processes.
2. **Map/Reduce Chains:** These chains involve mapping a function over a set of data and then reducing the results to a single output. They're great for parallel processing of large datasets.
3. **Router Chains:** These chains direct inputs to different sub-chains based on certain conditions, allowing for more complex, branching workflows.

1.3 Creating Custom Chains

Creating custom chains involves defining specific operations or functions that will be part of the chain. Here's an example of a custom sequential chain:

```
from langchain.chains import LLMChain
from langchain.llms import OpenAI
from langchain.prompts import PromptTemplate

class CustomChain:
    def __init__(self, llm):
        self.llm = llm
        self.steps = []

    def add_step(self, prompt_template):
        prompt = PromptTemplate(template=prompt_template, input_variables=["input"])
        chain = LLMChain(llm=self.llm, prompt=prompt)
        self.steps.append(chain)
```

```
def execute(self, input_text):
    for step in self.steps:
        input_text = step.run(input_text)
    return input_text

# Initialize the chain
llm = OpenAI(temperature=0.7)
chain = CustomChain(llm)

# Add steps to the chain
chain.add_step("Summarize the following text in one sentence: {input}")
chain.add_step("Translate the following English text to French: {input}")

# Execute the chain
result = chain.execute("LangChain is a powerful framework for building AI applications.")
print(result)
```

This example creates a custom chain that first summarizes an input text and then translates it to French.

2. Combining Chains and LLMs

2.1 Integrating Chains with Prompts and LLMs

Chains can be seamlessly integrated with prompts and LLMs to create more powerful and flexible systems. Here's an example:

```
from langchain import PromptTemplate, LLMChain
from langchain.llms import OpenAI
from langchain.chains import SimpleSequentialChain
```

```
llm = OpenAI(temperature=0.7)

# First chain: Generate a topic
first_prompt = PromptTemplate(
    input_variables=["subject"],
    template="Generate a random {subject} topic:"
)
first_chain = LLMChain(llm=llm, prompt=first_prompt)

# Second chain: Write a paragraph about the topic
second_prompt = PromptTemplate(
    input_variables=["topic"],
    template="Write a short paragraph about {topic}:"
)
second_chain = LLMChain(llm=llm, prompt=second_prompt)

# Combine the chains
overall_chain = SimpleSequentialChain(chains=[first_chain, second_chain], verbose=True)

# Run the chain
result = overall_chain.run("science")
print(result)
```

This example creates a chain that generates a random science topic and then writes a paragraph about it.

2.2 Debugging and Optimizing Chain-LLM Interactions

To debug and optimize chain-LLM interactions, you can use the `verbose` parameter and custom callbacks:

```
from langchain.callbacks import StdOutCallbackHandler
from langchain.chains import LLMChain
from langchain.llms import OpenAI
from langchain.prompts import PromptTemplate

class CustomHandler(StdOutCallbackHandler):
    def on_llm_start(self, serialized, prompts, **kwargs):
        print(f"LLM started with prompt: {prompts[0]}")

    def on_llm_end(self, response, **kwargs):
        print(f"LLM finished with response: {response.generations[0][0].text}")

llm = OpenAI(temperature=0.7, callbacks=[CustomHandler()])
template = "Tell me a {adjective} joke about {subject}."
prompt = PromptTemplate(input_variables=["adjective", "subject"], template=template)
chain = LLMChain(llm=llm, prompt=prompt, verbose=True)

result = chain.run(adjective="funny", subject="programming")
print(result)
```

This example uses a custom callback handler to provide detailed information about the LLM's input and output.

3. Introducing Agents

3.1 What are Agents in LangChain?

Agents in LangChain are autonomous entities that can use tools and make decisions to accomplish tasks. They combine LLMs with external tools to solve complex problems, allowing for more dynamic and adaptable AI systems.

3.2 Built-in Agents and Their Capabilities

LangChain provides several built-in agents, such as the zero-shot-react-description agent:

```
from langchain.agents import load_tools, initialize_agent, AgentType
from langchain.llms import OpenAI

llm = OpenAI(temperature=0)
tools = load_tools(["wikipedia", "llm-math"], llm=llm)

agent = initialize_agent(
    tools,
    llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)

result = agent.run("What is the square root of the year Plato was born?")
print(result)
```

This example creates an agent that can use Wikipedia and perform mathematical calculations to answer complex questions.

3.3 Creating Custom Agents

You can create custom agents by defining your own tools and agent classes. This allows for highly specialized agents tailored to specific tasks or domains.

Here's an example of a custom agent:

```
from langchain.agents import Tool, AgentExecutor, LLMSingleActionAgent
from langchain.prompts import StringPromptTemplate
from langchain import OpenAI, SerpAPIWrapper, LLMChain
from typing import List, Union
from langchain.schema import AgentAction, AgentFinish
import re

# Define custom tools
search = SerpAPIWrapper()
tools = [
    Tool(
        name="Search",
        func=search.run,
        description="Useful for answering questions about current events"
    )
]

# Define a custom prompt template
template = """Answer the following questions as best you can:

{input}

Use the following format:

Question: the input question you must answer
Thought: you should always think about what to do
Action: the action to take, should be one of [{tool_names}]
Action Input: the input to the action
Observation: the result of the action
... (this Thought/Action/Action Input/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question
```

Begin!

Question: {input}

Thought: To answer this question, I need to search for current information.
{agent_scratchpad}"""

```
class CustomPromptTemplate(StringPromptTemplate):
    template: str
    tools: List[Tool]

    def format(self, **kwargs) -> str:
        intermediate_steps = kwargs.pop("intermediate_steps")
        thoughts = ""
        for action, observation in intermediate_steps:
            thoughts += action.log
            thoughts += f"\nObservation: {observation}\nThought: "
        kwargs["agent_scratchpad"] = thoughts
        kwargs["tool_names"] = ", ".join([tool.name for tool in self.tools])
        return self.template.format(**kwargs)

prompt = CustomPromptTemplate(
    template=template,
    tools=tools,
    input_variables=["input", "intermediate_steps"]
)

# Define a custom output parser
class CustomOutputParser:
    def parse(self, llm_output: str) -> Union[AgentAction, AgentFinish]:
        if "Final Answer:" in llm_output:
            return AgentFinish(
                return_values={"output": llm_output.split("Final Answer:")[-1].strip()},
                log=llm_output,
            )
```

```
    action_match = re.search(r"Action: (\w+)", llm_output, re.DOTALL)
    action_input_match = re.search(r"Action Input: (.*)", llm_output, re.DOTALL)

    if not action_match or not action_input_match:
        raise ValueError(f"Could not parse LLM output: `{llm_output}`")

    action = action_match.group(1).strip()
    action_input = action_input_match.group(1).strip(" ").strip('\'')

    return AgentAction(tool=action, tool_input=action_input, log=llm_output)

# Create the custom output parser
output_parser = CustomOutputParser()

# Define the LLM chain
llm = OpenAI(temperature=0)
llm_chain = LLMChain(llm=llm, prompt=prompt)

# Define the custom agent
agent = LLMSingleActionAgent(
    llm_chain=llm_chain,
    output_parser=output_parser,
    stop=["\nObservation:"],
    allowed_tools=[tool.name for tool in tools]
)

# Create an agent executor
agent_executor = AgentExecutor.from_agent_and_tools(agent=agent, tools=tools, , verbose=True)
# Run the agent
result = agent_executor.run("What's the latest news about AI?")

print(result)
```

Conclusion

LangChain's chains and agents offer robust capabilities for constructing sophisticated AI-driven systems. When integrated with large language models (LLMs), they enable the creation of adaptable, smart applications designed to tackle a variety of tasks. As you progress through your LangChain journey, feel free to experiment with diverse chain types, agent setups, and custom modules to fully harness the framework's potential.

LangChian (4 Part Series)

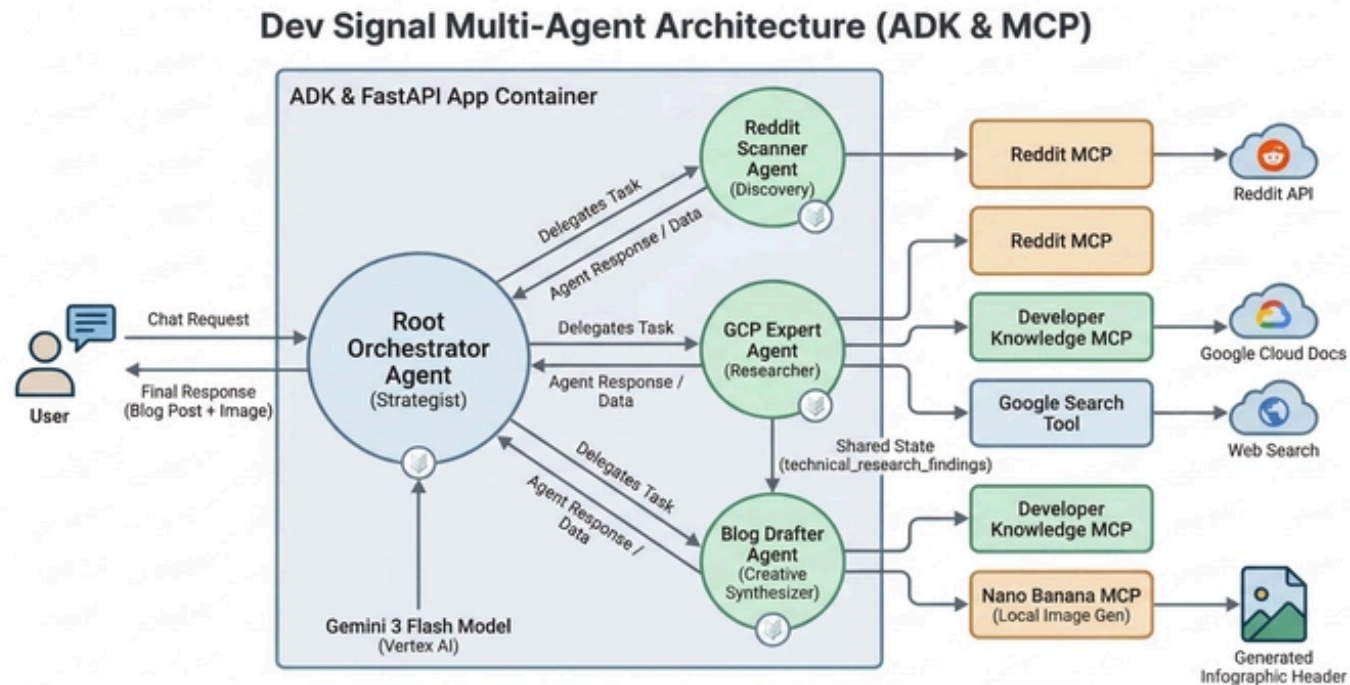
- 1 Mastering LangChain: Part 1 - Introduction to LangChain ...
- 2 Part 2: Mastering Prompts and Language Models with Lan...
- 3 Part 3: Building Powerful Chains and Agents in LangC...**
- 4 LangChain Part 4 - Leveraging Memory and Storage in La...



The DEV Team

PROMOTED





Architect A Personalized Multi-Agent System with Long-Term Memory

In support of our mission to accelerate the developer journey on Google Cloud, we built Dev Signal a multi-agent system designed to transform raw community signals into reliable technical

[Read More](#)

Top comments (3)



Oliver Parker • Jul 31 '24



Thanks for sharing. Have you checked other frameworks for building agents?



James 🌟 • Aug 5 '24



I've been thinking about doing a Series on LLama Index if that would be interesting to you. I can put the first part out sometime this week if you would be interested in that.



James 🌟 • Aug 5 '24



Here is a link to a LlamaIndex tutorial

dev.to/jamesbmour/llamaindex-revol...

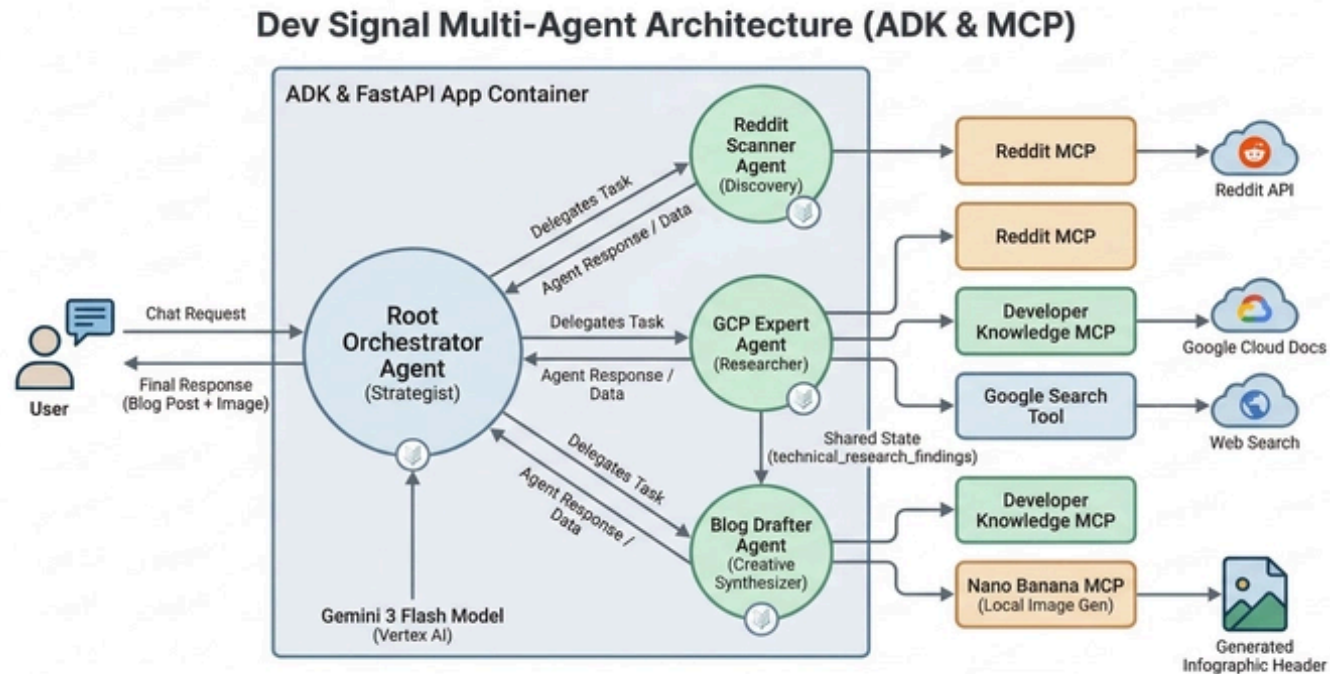
[Code of Conduct](#) • [Report abuse](#)



The DEV Team

PROMOTED





Architect A Personalized Multi-Agent System with Long-Term Memory

In support of our mission to accelerate the developer journey on Google Cloud, we built Dev Signal — a multi-agent system designed to transform raw community signals into reliable technical guidance by automating the path from discovery to expert creation.

[Read more →](#)

**James**

If you would like to support my writing or buy me a beer: <https://buymeacoffee.com/bmours>

LOCATION

Denver

EDUCATION

Ohio State - ISE , Georgia institute of technology - Masters in CS

WORK

Freelance Consulting

JOINED

Jan 23, 2024

More from James

Building a Human-in-the-Loop AI App with LangGraph and Ollama

#python #ai #llm #langgraph

Building a News and Stock Assistant with LangGraph and Chainlit

#programming #ai #tutorial #python

Mastering MCP Servers with LangChain and LangGraph: A Beginner's Guide

#python #programming #langgraph #ai



The DEV Team

PROMOTED





I built a native Android app in an afternoon, and I've never written a line of Kotlin

I've always thought building a mobile app required climbing a massive learning curve just to get a basic environment set up. To test that theory, I tried building my very first Android app using Google AI Studio.

Five minutes later, I had a working prototype.

[Read more →](#)



James

Posted on Aug 21, 2024



11

LangChain Part 4 - Leveraging Memory and Storage in LangChain: A Comprehensive Guide

#langchain #python #llm #tutorial

LangChain (4 Part Series)

1 Mastering LangChain: Part 1 - Introduction to LangChain ...

2 Part 2: Mastering Prompts and Language Models with Lan...

3 Part 3: Building Powerful Chains and Agents in LangChain

4 **LangChain Part 4 - Leveraging Memory and Storage i...**

LangChain Part 4 - Leveraging Memory and Storage in LangChain: A Comprehensive Guide

Code can be found here: [GitHub - jamesbmour/blog_tutorials](https://github.com/jamesbmour/blog_tutorials):

In the ever-evolving world of conversational AI and language models, maintaining context and efficiently managing information flow are critical components of building intelligent applications. LangChain, a powerful framework designed for working with large language models (LLMs), offers robust tools for memory management and data persistence, enabling the creation of context-aware systems.

In this guide, we'll delve into the nuances of leveraging memory and storage in LangChain to build smarter, more responsive applications.

1. Working with Memory in LangChain

Memory management in LangChain allows applications to retain context, making interactions more coherent and contextually relevant. Let's explore the different memory types and their use cases.

1.1. Types of Memory

LangChain provides various memory types to address different scenarios. Here, we'll focus on two key types:

ConversationBufferMemory

This memory type is ideal for short-term context retention, capturing and recalling recent interactions in a conversation.

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory()
memory.save_context({"input": "Hi, I'm Alice"}, {"output": "Hello Alice, how can I help you today?"})
memory.save_context({"input": "What's the weather like?"}, {"output": "I'm sorry, I don't have access to real-time data."})

print(memory.load_memory_variables({}))
```

ConversationSummaryMemory

For longer conversations, **ConversationSummaryMemory** is a great choice. It summarizes key points, maintaining context without overwhelming detail.

```
from langchain.memory import ConversationSummaryMemory
from langchain.llms import Ollama

llm = Ollama(model='phi3', temperature=0)
memory = ConversationSummaryMemory(llm=llm)
memory.save_context({"input": "Hi, I'm Alice"}, {"output": "Hello Alice, how can I help you today?"})
memory.save_context({"input": "I'm looking for a good Italian restaurant"}, {"output": "Great! I can help with that."})

print(memory.load_memory_variables({}))
```

1.2. Choosing the Right Memory Type for Your Use Case

Selecting the appropriate memory type depends on several factors:

- **Duration and Complexity:** Short sessions benefit from detailed context retention with `ConversationBufferMemory`, while long-term interactions may require summarization via `ConversationSummaryMemory`.
- **Detail vs. Overview:** Determine whether detailed interaction history or high-level summaries are more valuable for your application.
- **Performance:** Consider the trade-offs between the memory size and retrieval speed.

Use Cases:

- **ConversationBufferMemory:** Ideal for quick customer support or FAQ-style interactions.
- **ConversationSummaryMemory:** Best suited for long-term engagements like project management or ongoing customer interactions.

1.3. Integrating Memory into Chains and Agents

Memory can be seamlessly integrated into LangChain chains and agents to enhance conversational capabilities.

```
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory
# llm = OpenAI(temperature=0)
memory = ConversationBufferMemory()
conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

conversation.predict(input="Hi, I'm Alice")
conversation.predict(input="What's my name?")
```

This example illustrates how **ConversationBufferMemory** can be used to remember previous interactions, enabling more natural conversations.

2. Persisting and Retrieving Data

Persistent storage ensures that conversation history and context are maintained across sessions, enabling continuity in interactions.

2.1. Storing Conversation History and State

For basic persistence, you can use file-based storage with JSON:

```
import json

class PersistentMemory:
    def __init__(self, file_path):
        self.file_path = file_path
        self.load_memory()

    def load_memory(self):
        try:
            with open(self.file_path, 'r') as f:
                self.chat_memory = json.load(f)
        except FileNotFoundError:
            self.chat_memory = {'messages': []}

    def save_memory(self):
        with open(self.file_path, 'w') as f:
            json.dump({'messages': self.chat_memory['messages']}, f)

# Usage
memory = PersistentMemory(file_path='conversation_history.json')
print(memory.chat_memory)
```

This method allows you to persist conversation history in a simple, human-readable format.

2.2. Integrating with Databases and Storage Systems

For more scalable and efficient storage, integrating with databases like SQLite is recommended:

```
import sqlite3

class SQLiteMemory:
```

```
def __init__(self, db_path):
    self.db_path = db_path
    self.conn = sqlite3.connect(db_path)
    self.create_table()

def create_table(self):
    cursor = self.conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS conversations
        (id INTEGER PRIMARY KEY, input TEXT, output TEXT)
    ''')
    self.conn.commit()

def save_context(self, inputs, outputs):
    cursor = self.conn.cursor()
    cursor.execute('INSERT INTO conversations (input, output) VALUES (?, ?)',
                    (inputs['input'], outputs['output']))
    self.conn.commit()

def load_memory_variables(self, inputs):
    cursor = self.conn.cursor()
    cursor.execute('SELECT input, output FROM conversations ORDER BY id DESC LIMIT 10')
    rows = cursor.fetchall()
    history = "\\n".join([f"Human: {row[0]}\\nAI: {row[1]}" for row in reversed(rows)])
    return {"history": history }

# Usage
memory = SQLiteMemory('conversation_history.db')

print(memory.load_memory_variables({}))
```

3 Optimizing Memory Usage and Performance

To ensure your application remains responsive, consider these optimization strategies:

- **Efficient Data Structures:** Use structures like `deque` for managing fixed-size buffers.
- **Caching Strategies:** Reduce database queries by implementing caching for frequently accessed data.
- **Data Pruning:** Regularly prune or summarize old data to maintain a manageable memory size.

Here's an example of a memory class with basic caching:

```
import time

class CachedSQLiteMemory(SQLiteMemory):
    def __init__(self, db_path, cache_ttl=60):
        super().__init__(db_path)
        self.cache = None
        self.cache_time = 0
        self.cache_ttl = cache_ttl

    def load_memory_variables(self, inputs):
        current_time = time.time()
        if self.cache is None or (current_time - self.cache_time) > self.cache_ttl:
            var = self.cache
            self.cache = super().load_memory_variables(inputs)
            self.cache_time = current_time
            return self.cache

memory = CachedSQLiteMemory('conversation_history.db', cache_ttl=30)
```

This implementation caches the results of database queries for a specified time, reducing the load on the database and improving performance for applications that frequently access memory data.

Conclusion

Effective memory management is a cornerstone of building intelligent, context-aware conversational AI applications. LangChain provides a flexible and powerful framework for managing memory, allowing developers to tailor memory types to specific use cases, implement persistent storage solutions, and optimize performance for large-scale applications.

By choosing the right memory type, integrating persistent storage, and leveraging advanced techniques such as custom memory classes and caching strategies, you can build sophisticated AI systems that maintain context, improve user experience, and operate efficiently even as the scale and complexity of interactions grow.

With these tools and techniques at your disposal, you are well-equipped to harness the full potential of LangChain in creating responsive, intelligent, and contextually aware AI applications. Whether you're developing customer support bots, virtual assistants, or complex conversational systems, mastering memory and storage in LangChain will be a key factor in your success.

If you would like to support my writing or buy me a beer:

<https://buymeacoffee.com/bmours>

LangChain (4 Part Series)

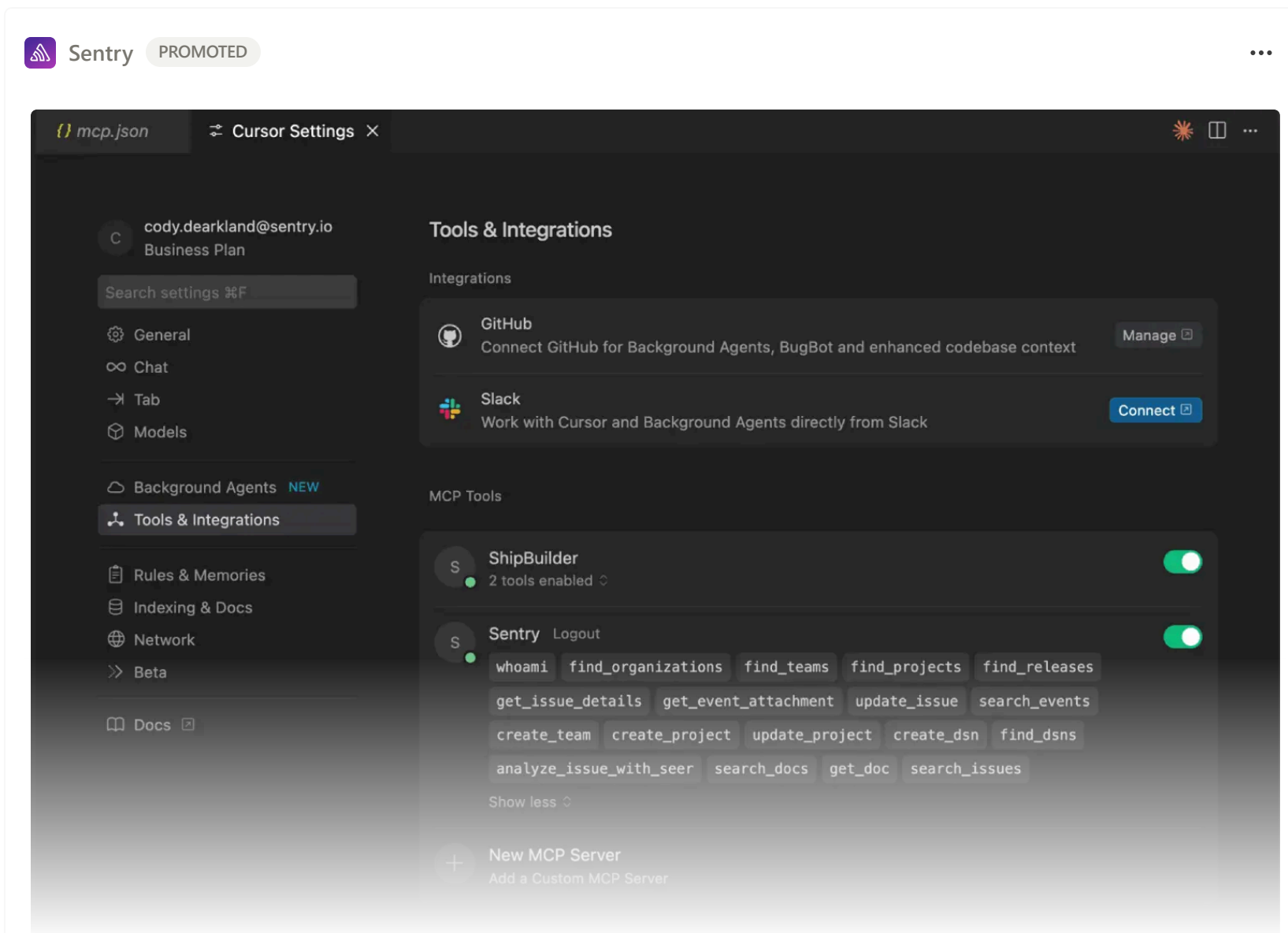
1 Mastering LangChain: Part 1 - Introduction to LangChain ...

2 Part 2: Mastering Prompts and Language Models with Lan...

3 Part 3: Building Powerful Chains and Agents in LangChain

4

LangChain Part 4 - Leveraging Memory and Storage i...



[Read More](#)

Top comments (0)

[Code of Conduct](#) • [Report abuse](#)



Bitrise

PROMOTED



A purple banner for Bitrise. In the top left is the Bitrise logo, which consists of a white icon of a face with a single eye and the word "bitrise" in white lowercase letters. Below the logo, the text "The CI/CD built for iOS." is written in a large, white, sans-serif font. Underneath that, "Ready for what's next" is written in a slightly smaller white font. Further down, the phrases "Build faster." and "Ship with confidence." are stacked in a smaller white font. At the bottom left of the banner is a white rounded rectangle containing the text "See it in action today" in a bold, dark blue font. On the right side of the banner is a 3D illustration of a purple smartphone standing on a white, stylized, interlocking ring structure. Various small icons are scattered around the phone and the ring, including a green square with a white rocket, a blue square with a white gear, and a yellow square with a white checkmark.

bitrise

The CI/CD built for iOS.

Ready for what's next

Build faster.
Ship with confidence.

See it in action today

Try Bitrise free and feel the DevOps difference today!

Your mobile team is growing but so is the chaos.

More people. More code. Endless moving parts. That's where Bitrise comes in. Our award-winning Mobile DevOps platform aligns your people, tools, and processes so you can build faster, ship sooner and reignite that joy in mobile dev again.

[Start a free trial](#)

**James**

If you would like to support my writing or buy me a beer: <https://buymeacoffee.com/bmours>

LOCATION

Denver

EDUCATION

Ohio State - ISE , Georgia institute of technology - Masters in CS

WORK

Freelance Consulting

JOINED

Jan 23, 2024

More from James

Building a Human-in-the-Loop AI App with LangGraph and Ollama

#python #ai #llm #langgraph

Building a News and Stock Assistant with LangGraph and Chainlit

#programming #ai #tutorial #python

Mastering MCP Servers with LangChain and LangGraph: A Beginner's Guide

#python #programming #langgraph #ai

**The DEV Team****PROMOTED**



I built a native Android app in an afternoon, and I've never written a line of Kotlin

I've always thought building a mobile app required climbing a massive learning curve just to get a basic environment set up. To test that theory, I tried building my very first Android app using Google AI Studio.

Five minutes later, I had a working prototype.

[Read more →](#)